

Démodulation

Javions – étape 4

1. Introduction

Le but de cette étape est d'écrire le code effectuant la « démodulation » des messages ADS-B, et de commencer la rédaction des classes représentant les différents types de messages.

2. Concepts

2.1. Messages ADS-B

Un message ADS-B est composé de 14 octets – 112 bits – dont le contenu est décrit dans la table suivante. Le CRC24 y est présenté comme faisant partie du message, mais il serait bien entendu possible de le considérer comme une donnée séparée.

De	À	Bits	Nom	Contenu
0	0	8 (5+3)	DF/CA	Format du message / capacité du transpondeur
1	3	24	ICAO	Adresse OACI de l'expéditeur
4	10	56	ME	Contenu (« charge utile »)
11	13	24	CRC	CRC24 du message

Le premier octet d'un message contient deux informations : son format (*downlink format*, abrégé DF) dans les 5 bits de poids fort, et la « capacité » (*capability*, abrégé CA) du transpondeur ayant envoyé le message dans les 3 bits de poids faible.

Le **transpondeur** (*transponder*) est l'appareil utilisé par l'aéronef pour envoyer les messages ADS-B, et son nom – une contraction de *transmitter* et *responder* – vient du fait qu'il est aussi capable de recevoir des messages.

La raison pour laquelle le transpondeur peut également recevoir des messages est que les contrôleurs aériens peuvent interroger à distance les aéronefs en leur envoyant des messages radio, auxquels les aéronefs répondent. Lorsqu'une telle communication bidirectionnelle a lieu entre un appareil situé au sol et un aéronef, le lien de communication entre le sol et l'aéronef est appelé *uplink*, tandis que le lien entre l'aéronef et le sol est appelé *downlink*.

Bien entendu, dans ce projet nous n'interrogerons pas les avions nous même, d'une part car cela est illégal, et d'autre part car la radio que nous utilisons ne peut pas émettre de signal. Nous nous contenterons donc de recevoir les messages que les aéronefs

transmettent spontanément.

(Il n'est pas très difficile d'interpréter également les messages que les avions envoient en réponse aux interrogations des contrôleurs aériens, mais nous avons choisi de ne pas le faire dans ce projet, afin de ne pas le compliquer excessivement.)

Le terme *extended squitter* est communément utilisé pour désigner les messages non sollicités que les aéronefs envoient pour diffuser des informations à leur sujet. Ces messages se reconnaissent au fait que leur format – c.-à-d. leur attribut DF – vaut 17, qui est le seul genre de message ADS-B que nous traiterons.

Le contenu réellement intéressant des messages est transmis dans leur attribut ME, qui comporte 7 octets (56 bits). Les données contenues dans cet attribut peuvent être de plusieurs types, et ses 5 bits de poids fort, qui constituent son code de type (*type code*), permettent de savoir ce que contiennent ses 51 bits restants. La signification exacte des bits de l'attribut ME sera examinée à partir de l'étape suivante.

2.2. Modulation

Pour transmettre les messages ADS-B par onde radio, une porteuse à 1090 MHz est modulée de la manière très simple décrite à l'étape précédente. Pour mémoire, la technique de modulation consiste à transmettre les bits du message l'un après l'autre, au rythme de 1 bit par μs , en :

- ne transmettant rien durant $\frac{1}{2} \mu\text{s}$ puis en transmettant la porteuse durant $\frac{1}{2} \mu\text{s}$ lorsque le bit à transmettre vaut 0,
- transmettant la porteuse durant $\frac{1}{2} \mu\text{s}$ puis ne transmettant rien durant $\frac{1}{2} \mu\text{s}$ lorsque le bit à transmettre vaut 1.

On nomme **impulsion** (*pulse*) une période de $\frac{1}{2} \mu\text{s}$ durant laquelle la porteuse est transmise. Chaque bit est donc représenté au moyen d'une impulsion, qui est placée soit dans la première, soit dans la seconde moitié de la microseconde dédiée à ce bit.

Telle quelle, cette technique de modulation pose toutefois un problème de synchronisation entre l'émetteur et le récepteur, car ce dernier ne peut pas savoir quand un message débute. En effet, s'il détecte une impulsion, il n'a aucun moyen de déterminer si elle correspond à un premier bit valant 0 – auquel cas la transmission du message aurait déjà commencé depuis $\frac{1}{2} \mu\text{s}$ – ou à un premier bit valant 1 – auquel cas la transmission du message commencerait avec cette impulsion.

Pour résoudre ce problème, un message ADS-B est toujours précédé d'un **préambule** (*preamble*) d'une durée de $8 \mu\text{s}$, qui consiste en quatre impulsions transmises après 0, 1, $3\frac{1}{2}$ et $4\frac{1}{2} \mu\text{s}$. Le préambule a donc une durée égale à celle nécessaire à la transmission de 8 bits, mais il ne peut pas être interprété comme une séquence de 8 bits. En effet, la période de « silence » entre la fin de la seconde impulsion à $1.5 \mu\text{s}$ et le début de la troisième à $3.5 \mu\text{s}$ ne peut pas se produire lors de la transmission des bits d'un message. La forme exacte de ce préambule étant fixe, un récepteur peut l'utiliser pour déterminer

sans ambiguïté où commence un message ADS-B.

La durée totale nécessaire à la transmission d'un message ADS-B complet – préambule compris – est de $120\ \mu\text{s}$ puisque le préambule nécessite $8\ \mu\text{s}$ tandis que les 112 bits nécessitent chacun $1\ \mu\text{s}$.

2.3. Démodulation

→ "décoder un signal"

L'opération consistant à déterminer les bits d'un message à partir du signal fourni par une radio se nomme **démodulation** (*demodulation*). Par léger abus de langage, nous utiliserons également ce terme pour désigner l'opération consistant à trouver tous les messages présents dans un signal radio, et à les démoduler les uns après les autres.

Dans le cas des messages ADS-B, cela peut se faire en recherchant le préambule d'un message dans le signal radio, et dès sa détection, à décoder les bits du message en déterminant, pour chacune des périodes de $1\ \mu\text{s}$ correspondant à l'un d'eux, si l'impulsion se trouve dans sa première ou dans sa seconde moitié.

Pour effectuer cette recherche, nous nous baserons sur les échantillons de puissance calculés précédemment. Comme ces échantillons sont espacés de $0.1\ \mu\text{s}$, un message ADS-B complet – préambule compris – correspond à 1200 d'entre eux.

Dès lors, pour trouver tous les messages ADS-B contenus dans un signal, on peut commencer par regarder si l'un d'entre eux se trouve dans les 1200 premiers échantillons de puissance, d'index 0 à 1199. Si tel est le cas, alors le message peut être démodulé et la recherche peut ensuite continuer à partir de l'échantillon d'index 1200. Par contre, si aucun message ne commence à l'échantillon d'index 0, on poursuit la recherche avec le prochain échantillon, en regardant si un message se trouve dans les échantillons d'index 1 à 1200. Et ainsi de suite.

En d'autres termes, la recherche de tous les messages ADS-B contenus dans une séquence d'échantillons peut se faire en déplaçant une fenêtre, d'une taille de 1200 échantillons, sur cette séquence. Lorsqu'on constate la présence d'un préambule au début de la fenêtre, on en conclut qu'elle contient un message, que l'on démodule. Ensuite, la recherche se poursuit en avançant la fenêtre, soit de 1200 échantillons si elle contient un message, soit de 1 échantillon si elle n'en contient pas.

2.3.1. Recherche du préambule

Pour déterminer si un préambule commence au début de la fenêtre, on calcule deux sommes d'échantillons de puissance, qui sont :

1. celle des échantillons situés à $0, 1, 3\frac{1}{2}$ et $4\frac{1}{2}\ \mu\text{s}$ du début de la fenêtre, qui correspondent aux périodes durant lesquelles la porteuse devrait être émise,
2. celle des échantillons situés à $\frac{1}{2}, 1\frac{1}{2}, 2, 2\frac{1}{2}, 3$ et $4\ \mu\text{s}$ du début de la fenêtre, qui correspondent à une partie des périodes durant lesquelles la porteuse ne devrait pas être émise.

Ensuite, ces deux sommes sont comparées, et si la première est au moins égale au double de la seconde, alors on considère qu'un préambule de message ADS-B se trouve bien au début de la fenêtre.

Plus formellement, si on note w_i l'échantillon de puissance d'index i dans la fenêtre, les formules permettant calculer les deux sommes ci-dessus sont :

$$\begin{aligned}\Sigma_p &= w_0 + w_{10} + w_{35} + w_{45} \\ \Sigma_v &= w_5 + w_{15} + w_{20} + w_{25} + w_{30} + w_{40}\end{aligned}$$

où Σ_p est la somme des « pics » – là où la porteuse devrait être transmise – et Σ_v la somme des « vallées » – là où la porteuse ne devrait pas être transmise.

Une fois ces deux sommes calculées, on considère qu'un préambule commence bien au début de la fenêtre si et seulement si $\Sigma_p \geq 2\Sigma_v$. Cette condition a été déterminée de manière empirique, et le facteur 2 utilisé n'a donc pas de signification particulière.

2.3.2. Alignement de la fenêtre

Jusqu'à présent, nous avons fait l'hypothèse que nous désirions aligner la fenêtre avec le début du préambule d'un message ADS-B. Or en pratique il peut être préférable de l'aligner autant que possible avec le *centre* de la première impulsion du préambule, c.-à-d. $\frac{1}{4} \mu s$ après le début du message.

En effet, en l'alignant ainsi, toutes les impulsions du message ou de son préambule seront centrées autour des multiples de $0.5 \mu s$ depuis le début de la fenêtre – donc autour des multiples de 5 échantillons. Or c'est au centre d'une impulsion que la puissance du signal est la plus forte, et donc c'est là que sa valeur doit être mesurée pour déterminer de manière aussi fiable que possible si la porteuse était transmise durant cette période de $0.5 \mu s$ ou non.

pour venir analyser le signal sur les pics ?

Il n'existe pas de méthode infaillible pour aligner la fenêtre sur le milieu de la première impulsion du préambule, mais nous utiliserons une technique qui semble bien fonctionner en pratique et qui consiste à considérer qu'un préambule ne peut commencer à une position donnée que si elle correspond à un maximum local de la somme Σ_p .

on va utiliser cette méthode

En d'autres termes, la somme Σ_p est calculée pour toutes les positions de la fenêtre, mais si la valeur correspondant à sa position actuelle n'est pas strictement plus grande que celle correspondant aux positions précédente et suivante, alors on considère que la fenêtre ne contient pas de préambule.

Plus formellement, si on note $\Sigma_{p,0}$ la valeur de cette somme correspondant à la fenêtre actuelle, $\Sigma_{p,-1}$ celle correspondant à la position précédente de la fenêtre (un échantillon plus à gauche) et $\Sigma_{p,+1}$ celle correspondant à la position suivante de la fenêtre (un échantillon plus à droite), alors on ne considère qu'un préambule commence bien au début de la fenêtre que si :

$$\Sigma_{p,-1} < \Sigma_{p,0} \quad \text{et} \quad \Sigma_{p,0} > \Sigma_{p,+1} \quad \text{et} \quad \underbrace{\Sigma_{p,0} \geq 2\Sigma_v}_{\text{✓ ou avant}}$$

La dernière de ces trois conditions est celle déjà mentionnée à la section précédente.

2.3.3. Décodage des bits

Une fois la fenêtre alignée sur ce que l'on suppose être le centre de la première impulsion du préambule d'un message ADS-B, la valeur des bits qui le composent peut être déterminée très facilement.

En effet, le bit d'index i , noté b_i , est déterminé en comparant la puissance du signal au centre des deux périodes de $0.5 \mu s$ durant lesquelles l'impulsion pourrait avoir été transmise :

$$b_i = \begin{cases} 0 & \text{si } w_{80+10i} < w_{85+10i} \\ 1 & \text{sinon} \end{cases}$$

→ décodage des bits

où 80 correspond au nombre d'échantillons constituant le préambule.

2.3.4. Validation des messages

Lors de la démodulation d'un message, certaines vérifications doivent être faites pour s'assurer d'une part qu'il s'agit d'un message qui nous intéresse, et d'autre part qu'il a été démodulé correctement.

→ validation / vérification

Pour ce faire, après avoir démodulé le premier octet, on commence par vérifier que le format du message – son attribut DF – correspond bien au type 17 qui nous intéresse.

Cette vérification est importante, car les aéronefs transmettent d'autres messages débutant par un préambule identique à celui décrit plus haut, mais avec un autre format que nous ne gérons pas.

Si le format correspond à nos attentes, alors la totalité des 112 bits du message peuvent être démodulés et le CRC24 attaché au message vérifié. Les messages pour lesquels il n'est pas valide sont simplement ignorés. Une autre option serait d'essayer de détecter et de corriger les erreurs n'affectant qu'un seul bit, mais dans un souci de simplicité, nous ne le ferons pas.

3. Mise en œuvre Java

La mise en œuvre de cette étape consiste en une classe représentant les messages ADS-B « bruts », une classe représentant le démodulateur, et deux interfaces qui ne seront utiles que dans des étapes ultérieures mais qui ont été intégrées à celle-ci pour des questions d'organisation.

3.1. Enregistrement RawMessage

L'enregistrement `RawMessage` du sous-paquetage `adsb`, public, représente un message ADS-B « brut », c.-à-d. dont l'attribut `ME` n'a pas encore été analysé. Il possède les attributs suivants :

- `long timeStampNs`, l'horodatage du message, exprimé en nanosecondes depuis une origine donnée – généralement l'instant correspondant au tout premier échantillon de puissance calculé,
- `ByteString bytes`, les octets du message.

Le constructeur compact de `RawMessage` lève `IllegalArgumentException` si l'horodatage est (strictement) négatif, ou si la chaîne d'octets ne contient pas `LENGTH` octets, où `LENGTH` est la constante suivante, stockée comme d'habitude dans un attribut public, statique et final de la classe :

- `int LENGTH`, qui vaut 14, la longueur en octets des messages ADS-B.

`RawMessage` offre de plus les méthodes statiques suivantes :

- `RawMessage of(long timeStampNs, byte[] bytes)`, qui retourne le message ADS-B brut avec l'horodatage et les octets donnés, ou `null` si le CRC24 des octets ne vaut pas 0,
- `int size(byte byte0)`, qui retourne la taille d'un message dont le premier octet est celui donné, et qui vaut `LENGTH` si l'attribut `DF` contenu dans ce premier octet vaut 17, et 0 sinon – indiquant que le message n'est pas d'un type connu,
- `static int typeCode(long payload)`, qui retourne le code de type de l'attribut `ME` passé en argument.

Finalement, `RawMessage` offre les méthodes publiques suivantes :

- `int downLinkFormat()`, qui retourne le format du message, c.-à-d. l'attribut `DF` stocké dans son premier octet,
- `IcaoAddress icaoAddress()`, qui retourne l'adresse OACI de l'expéditeur du message,
- `long payload()`, qui retourne l'attribut `ME` du message – sa « charge utile »,
- `int typeCode()`, qui retourne le code de type du message, c.-à-d. les cinq bits de poids le plus fort de son attribut `ME`.

3.2. Classe `AdsbDemodulator`

La classe `AdsbDemodulator` du sous-paquetage `demodulation`, publique et finale, représente un démodulateur de messages ADS-B. Elle offre un unique constructeur public :

- `AdsbDemodulator(InputStream samplesStream)` throws `IOException`, qui retourne un démodulateur obtenant les octets contenant les échantillons du flot passé en argument ; lève `IOException` si une erreur

d'entrée/sortie se produit lors de la création de l'objet de type `PowerWindow` représentant la fenêtre de 1200 échantillons de puissance, utilisée pour rechercher les messages.

En dehors de ce constructeur, `AdsbDemodulator` n'offre qu'une seule méthode publique :

- `RawMessage nextMessage()` throws `IOException`, qui retourne le prochain message ADS-B du flot d'échantillons passé au constructeur, ou `null` s'il n'y en a plus – c.-à-d. que la fin du flot d'échantillons a été atteinte ; lève `IOException` en cas d'erreur d'entrée/sortie.

L'horodatage des messages retournés par `nextMessage` utilise l'instant correspondant au tout premier échantillon de puissance comme origine du temps.

3.3. Interface `Message`

L'interface `Message` du sous-paquetage `adsb`, publique, a pour but d'être implémentée par toutes les classes représentant des messages ADS-B « analysés », qui seront écrites à partir de l'étape suivante.

Cette interface est très simple et ne possède que deux méthodes, publiques et abstraites, qui sont :

- `long timeStampNs()`, qui retourne l'horodatage du message, en nanosecondes,
- `IcaoAddress icaoAddress()`, qui retourne l'adresse OACI de l'expéditeur du message.

Ces méthodes étant abstraites, il n'est bien entendu pas possible d'écrire de test unitaire pour elles.

3.4. Interface `AircraftStateSetter`

L'interface `AircraftStateSetter` du sous-paquetage `adsb`, publique, a pour but d'être implémentée par toutes les classes représentant l'état (modifiable) d'un aéronef.

Cette interface n'est pas utile dans le cadre de cette étape, et elle ne sera implémentée que par une classe que nous réaliserons à l'étape 6. Son utilité peut donc être difficile à percevoir à ce stade, même si le but de chacune de ses méthodes devrait être clair. Chacune d'elles n'est en effet rien d'autre qu'un « modificateur » (*setter*) permettant de changer la valeur d'un attribut particulier de l'état d'un aéronef.

Ces méthodes, toutes abstraites, sont :

- `void setLastMessageTimeStampNs(long timeStampNs)`, qui change l'horodatage du dernier message reçu de l'aéronef à la valeur donnée,
- `void setCategory(int category)`, qui change la catégorie de l'aéronef à la

- valeur donnée (le concept de catégorie d'un aéronef sera décrit dans une étape ultérieure),
- `void setCallSign(CallSign callSign)`, qui change l'indicatif de l'aéronef à la valeur donnée,
- `void setPosition(GeoPos position)`, qui change la position de l'aéronef à la valeur donnée,
- `void setAltitude(double altitude)`, qui change l'altitude de l'aéronef à la valeur donnée,
- `void setVelocity(double velocity)`, qui change la vitesse de l'aéronef à la valeur donnée,
- `void setTrackOrHeading(double trackOrHeading)`, qui change la direction de l'aéronef à la valeur donnée.

Tout comme l'interface `Message`, l'interface `AircraftStateSetter` ne contient que des méthodes abstraites, et il n'est donc pas possible de la tester.

3.5. Tests

Comme d'habitude, nous ne vous fournissons plus de tests mais [un fichier de vérification de signatures](#) à importer dans votre projet.

Pour que vous puissiez tester votre démodulateur, nous mettons de plus à votre disposition [une archive Zip](#) contenant un fichier d'échantillons similaire à celui fourni à l'étape précédente, mais beaucoup plus volumineux.

Ce fichier porte le nom `samples_20230304_1442.bin` car il a été enregistré le 4 mars 2023 à 14h42, et contient 150 millions d'échantillon, ce qui correspond à une durée d'enregistrement de 7½ secondes. En plaçant ce fichier dans le répertoire de votre projet puis en exécutant le programme ci-dessous, vous devriez voir les messages ADS-B contenus dans le fichier affichés à l'écran.

```
public final class PrintRawMessages {
    public static void main(String[] args) throws IOException {
        String f = "samples_20230304_1442.bin";
        try (InputStream s = new FileInputStream(f)) {
            AdsbDemodulator d = new AdsbDemodulator(s);
            RawMessage m;
            while ((m = d.nextMessage()) != null)
                System.out.println(m);
        }
    }
}
```

Au total, 384 messages devraient être affichés, les cinq premiers étant :

```
RawMessage[timeStampNs=8096200, bytes=8D4B17E5F8210002004BB8B1F1AC]
```



```
RawMessage[timestampNs=75898000, bytes=8D49529958B302E6E15FA352306B]
RawMessage[timestampNs=100775400, bytes=8D39D300990CE72C70089058AD77]
RawMessage[timestampNs=116538700, bytes=8D4241A9601B32DA4367C4C3965E]
RawMessage[timestampNs=129268900, bytes=8D4B1A00EA0DC89E8F7C0857D5F5]
```

Le premier d'entre eux provient de l'aéronef dont l'adresse OACI est [4B17E5](#), le second de celui dont l'adresse est [495299](#), etc.

Ce programme vous permet également de tester les performances du code que vous avez écrit jusqu'à présent. Sachant que les données du fichier correspondent à 7½ secondes, il faudrait que le programme prenne (bien) moins que ce temps-là pour afficher la totalité des messages, faute de quoi il ne serait pas assez rapide pour traiter en temps réel les données provenant d'une AirSpy.

Pour vous donner une idée, avec le code du corrigé, le programme ci-dessus affiche la totalité des messages en un peu moins de 1½ secondes lorsqu'on l'exécute sur un MacBook Pro datant de 2019.

4. Résumé

Pour cette étape, vous devez :

- écrire les classes et interfaces `RawMessage`, `AdsbDemodulator`, `Message` et `AircraftStateSetter` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies,
- rendre votre code au plus tard le **17 mars 2023 à 18h00**, au moyen du programme `Submit.java` fourni et des jetons disponibles sur votre [page privée](#).

Ce rendu est un rendu testé, auquel 18 points sont attribués, au prorata des tests unitaires passés avec succès.

N'attendez surtout pas le dernier moment pour effectuer votre rendu, car vous n'êtes pas à l'abri d'imprévus. **Souvenez-vous qu'aucun retard, aussi insignifiant soit-il, ne sera toléré !**